

AXIS · SECURITY REVIEW

Axis
periphery.

A security review of periphery.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Axis	periphery	Guardian	July 23 – August 17, 2024	August 26, 2024

§ 00

Contents.

51 pages — overview, confidence ranking, scope, 35 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 28 findings	P. 10
	Remediation Review · 7 findings	P. 42
§ 05	Disclaimer	P. 50
§ 06	About Guardian	P. 51

§ 01

Overview.

Guardian reviewed periphery across 2 review rounds. The final report includes 2 Critical, 9 High, 7 Medium, 17 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	Axis-Fi/axis-periphery
CLIENT	METHODOLOGY
Axis	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
periphery	9da756b9662e
PREPARED BY	
Guardian	
AUDIT WINDOW	
July 23 – August 17, 2024	
FINAL REPORT	
August 26, 2024	

1.1 Confidence Ranking

2 / 5

Low Confidence

Guardian assigns a Confidence Ranking of 2 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
■ Critical	2	0	0	0	2
■ High	9	0	1	1	7
■ Medium	7	0	4	0	3
■ Low	17	0	8	0	9
■ Informational	0	0	0	0	0
All severities	35	0	13	1	21

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence ASSIGNED	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed periphery in `Axis-Fi/axis-periphery`. The scope includes 615 source lines across 5 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/callbacks/liquidity/UniswapV3DTL.sol	110	224
src/callbacks/liquidity/UniswapV2DTL.sol	55	122
src/callbacks/liquidity/BaseDTL.sol	192	385
src/callbacks/liquidity/BaselineV2/BaselineAxisLaunch.sol	210	523
dependencies/axis-core-1.0.0/src/bases/BaseCallback.sol	48	192

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Gaming Uni V3 Initialization	DoS	CRITICAL	RESOLVED	p. 10
C-02	Uniswap V2 Auction Settlement DoS	Logical Error	CRITICAL	RESOLVED	p. 12
H-01	Curator Fees Will DoS Auction Settlement	Logical Error	HIGH	RESOLVED	p. 14
H-02	Pool Percentage Causes DoS	DoS	HIGH	RESOLVED	p. 15
H-03	Baseline Settlement DoS Via External Liquidity	DoS	HIGH	RESOLVED	p. 16
H-04	bAsset Price Set Below Floor	Gaming	HIGH	PARTIALLY RESOLVED	p. 17
H-05	Malicious Sellers Can Steal Auction Proceeds	Gaming	HIGH	RESOLVED	p. 18
H-06	Anchor Width Param Will DoS Auction Settlement	DoS	HIGH	RESOLVED	p. 19
H-07	Lot Creation DoS Through Uniswap Pair	DoS	HIGH	RESOLVED	p. 20
H-08	Baseline Launches Can Not Be Done	DoS	HIGH	RESOLVED	p. 21
M-01	discoveryTickWidth Configured Too Low	Logical Error	MEDIUM	RESOLVED	p. 22
M-02	Inconsistent Prices Between Pool and Auction	Validation	MEDIUM	RESOLVED	p. 24
M-03	Incorrect Liquidity Structure Deployed	Logical Error	MEDIUM	RESOLVED	p. 25

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-04	Auction Settlement DoS'd By Creator	DoS	MEDIUM	ACKNOWLEDGED	p. 26
M-05	Blacklisted Addresses Halt Auction Settlement	DoS	MEDIUM	ACKNOWLEDGED	p. 27
M-06	Outdated Vesting Modules Used	Logical Error	MEDIUM	ACKNOWLEDGED	p. 28
L-01	console.log Present in Code	Best Practices	LOW	ACKNOWLEDGED	p. 29
L-02	slide Inoperable Due To Floor Config	Validation	LOW	RESOLVED	p. 30
L-03	Early Unvesting Is Possible	Logical Error	LOW	RESOLVED	p. 31
L-04	Unused active Flag	Validation	LOW	RESOLVED	p. 33
L-05	Discovery Range Loose Validations	Validation	LOW	ACKNOWLEDGED	p. 34
L-06	Unused Error	Best Practices	LOW	RESOLVED	p. 35
L-07	High floorReservesPercent Causes DoS	Validation	LOW	RESOLVED	p. 36
L-08	Arbitrary Byte Length Risk	Warning	LOW	RESOLVED	p. 37
L-09	Esoteric Token Pairs Are Not Supported	Warning	LOW	ACKNOWLEDGED	p. 38
L-10	Launches Restrict Liquidity Structure	Validation	LOW	RESOLVED	p. 39
L-11	Misleading Comment	Best Practices	LOW	RESOLVED	p. 40
L-12	Seller May Receive All Proceeds	Logical Error	LOW	RESOLVED	p. 41

REMEDIATION REVIEW

H-02	Capacity Validation Doesn't Account For Fees	Logical Error	HIGH	ACKNOWLEDGED	p. 42
M-01	Baseline Launch Create DoS	DoS	MEDIUM	ACKNOWLEDGED	p. 43
L-01	Uniswap V2 Auctions Can Be DoS'd With Many Quote Tokens	Warning	LOW	ACKNOWLEDGED	p. 45

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-02	Lacking safeTransfer Usage	Best Practices	LOW	ACKNOWLEDGED	p. 46
L-03	Unnecessary totalCollatSupply	Optimization	LOW	ACKNOWLEDGED	p. 47
L-04	Missing auctionComplete Check	Validation	LOW	ACKNOWLEDGED	p. 48
L-05	Loop Vault Capacity Is Not Included	Warning	LOW	ACKNOWLEDGED	p. 49

CRITICAL **C-01**

RESOLVED

Gaming Uni V3 Initialization

CATEGORY	DoS
LOCATION	UniswapV3DTL::L193&205
REVIEW	Main Review

DESCRIPTION

Through initializing the Uniswap V3 pool, a malicious user can manipulate the price of the tokens from an auction and sell their newly purchased tokens for a profit. This attack is possible because any person can call `settle()` and set the `maxSlippage` to an unreasonably high number. In order to execute this attack, a malicious user would need to:

1. Buy tokens in an auction.
2. Initialize a Uniswap V3 Pool with a high price.
3. Call `settle()` with `maxSlippage` set to `100e2`.
4. Sell their tokens in the Uniswap V3 Pool.

It is also worth noting that if `settle()` is first called by the seller with a reasonable slippage tolerance, the transaction will revert, and the attacker will be able to follow up and execute the transaction with the proper slippage.

RECOMMENDATION

There are two possible solutions for this issue. The first is to make `settle()` only callable by the seller, so that they can specify the slippage tolerance. Then wrap the callback in a `try/catch` block. Inside the `catch`, transfer the tokens back to the seller if the transaction reverts.

Alternatively if you do not wish to make changes to the core system, add `minSlippageAmtOutToken0` and `minSlippageAmtOutToken1` to `DTLConfiguration`. This way the owner can be certain that slippage is not set to an intolerable amount, regardless of who calls `settle()`.

PROOF OF CONCEPT https://github.com/GuardianAudits/axis-1/blob/Gameing_V3_Init/test/guardian/PoCs/GuradianPoCGameV3.t.sol

RESOLUTION

Status: Resolved.

C-01 Gaming Uni V3 Initialization

CONTINUED

<https://github.com/Axis-Fi/axis-periphery/commit/04e73e93181019261cd0c8d0f425b4f526fcc860>

CRITICAL **C-02**

RESOLVED

Uniswap V2 Auction Settlement DoS

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/axis-2/blob/master/src/callbacks/liquidity/UniswapV2DTL.sol#L119
REVIEW	Main Review

— DESCRIPTION

When a UniV2 pool is used as a part of the token launch the settle function will attempt to add liquidity to the pair. However this will only be successful if the pool has either a non-zero balance of both assets. Or a zero balance of both assets.

If the pair has a non-zero reserve for one asset and a zero reserve for the other it will revert when attempting to add liquidity.

This is due to the following check when the `addLiquidity` function calls the `quote` function.

```
require(reserveA > 0 && reserveB > 0, 'UniswapV2Library: INSUFFICIENT_LIQUIDITY');
```

Typically it is not possible to get a pair into this state as the first time adding liquidity to a pair, both assets must be added.

However an attacker can bypass this by donating a dust amount of one of the assets to the pair and then calling `sync`. By doing this the reserves of that pair will update and to having one reserve being zero and the other being non-zero. Thus causing the revert when the auction tries to settle.

All an attacker would need to do in this case is monitor Axis auction creations, create the pair ahead of time and donate 1 wei of the quote token. By doing this the auction will not be able to settle.

— RECOMMENDATION

Calculate the amount to be minted to the pair and call the `mint` function instead of `addLiquidity` to avoid this DOS case.

PROOF OF CONCEPT <https://gist.github.com/devNamedKiki/6d3c4b07bde8413be6f2ccd418662d6c>

C-02 Uniswap V2 Auction Settlement DoS

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/pull/13>

HIGH **H-01**

RESOLVED

Curator Fees Will DoS Auction Settlement

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/axis-1/blob/master/src/callbacks/liquidity/BaselineV2/BaselineAxisLaunch.sol#L456
REVIEW	Main Review

— DESCRIPTION

Auction seller can opt to set a curator in charge of approving a batch auction lot. This curator will receive some base tokens.

The issue relies on Baseline auctions, as curator fees were not expected to be enabled, but fees in bAssets are actually minted during the `onCurate` callback, as this permission flag is enabled in the callback contract.

This additional bAssets minted will cause the `onSettle` callback to revert, as curator fees increases the bAsset spot supply, breaking the capacity invariant (capacity ratio < 100%) and causing a DoS on auction settlement

— RECOMMENDATION

If curator fees are enabled for Baseline Auctions, consider accounting for these fees during `onSettle` callback. Otherwise, revert during the `onCurate` callback.

PROOF OF CONCEPT <https://github.com/GuardianAudits/axis-1/blob/guardian/test/guardian/tests/GuardianTestBaseline.t.sol#L974>

— RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/pull/11/commits/47a44a0540ab17153473744f789a02d77dabc742>

HIGH **H-02**

RESOLVED

Pool Percentage Causes DoS

CATEGORY	DoS
LOCATION	BaselineAxisLaunch::L313
REVIEW	Main Review

DESCRIPTION

For Baseline, allowing a user to set the `poolPercent`, can cause a revert in `_onSettle()`. If the `poolPercent` is not configured to 100%, the revert will occur due to the `capacityRatio` not being within a tolerable range. The `capacityRatio` is not satisfied because there is not enough backing liquidity to support the Baseline system. Since the revert is not caught in `_onCreate()`, an auction will be carried out, but then it will become unseizable.

RECOMMENDATION

Do not allow the user to configure `poolPercent`, it should be set to 100%.

PROOF OF CONCEPT <https://github.com/GuardianAudits/axis-1/blob/guardian/test/guardian/tests/GuardianTestBaseline.t.sol#L1040>

RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/pull/11/commits/47a44a0540ab17153473744f789a02d77dabc742>

HIGH **H-03**

RESOLVED

Baseline Settlement DoS Via External Liquidity

CATEGORY	DoS
LOCATION	https://github.com/GuardianAudits/axis-2/blob/c4ed61098bf01827a2519ee0048b81e307c4280c/src/callbacks/liquidity/BaselineV2/BaselineAxisLaunch.sol#L592C
REVIEW	Main Review

— DESCRIPTION

Reserves added as liquidity during Baseline token launches must have enough capacity to buy back all circulating `bAsset`s at the floor price. This verification is conducted in the `_onSettle` function, and the total capacity must fall within the range of 100% to 102%.

The calculation of capacity involves the total liquidity amounts of the BPOOL contract in floor, anchor, and discovery ranges. However, anyone can add liquidity on behalf of the BPOOL before settlement, increasing the total capacity beyond 102% and causing the `_onSettle` callback to fail.

— RECOMMENDATION

It is suggested to verify the LP token balance of the BPOOL before adding reserves/liquidity to the underlying pool and burn these tokens. Alternatively, consider allowing any capacity ratio above 100% without restricting it to 102%.

PROOF OF CONCEPT <https://github.com/GuardianAudits/axis-2/pull/3>

— RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/pull/11/commits/3e4061c7d1fd6f027f5b8ecb95e5e0398785e3dd>

HIGH **H-04**

PARTIALLY RESOLVED

bAsset Price Set Below Floor

CATEGORY Gaming
LOCATION Global
REVIEW Main Review

— DESCRIPTION

The Uniswap pool is set up and initialized with the BPOOL contract before the auction begins.

The pool will initially have 0 liquidity, as a result, a malicious actor may trigger a swap with 0 bAsset input amount and a priceLimit to set the price where they wish.

The pool price can then be assigned to below the floor tick, perturbing the launch of the system.

— RECOMMENDATION

Consider implementing logic in the `_onSettle` callback to re-adjust the price back to the upper tick of the anchor range by swapping with 1 wei reserves in with a priceLimit at the desired price to assign the price to the bottom of the upper tick range of the Anchor.

PROOF OF CONCEPT <https://github.com/GuardianAudits/axis-1/pull/4>

— RESOLUTION

Status: Partially resolved.

<https://github.com/Axis-Fi/axis-periphery/pull/11/commits/d37f1b29607ee455334d1b60775154bf795a460a>

HIGH **H-05**

RESOLVED

Malicious Sellers Can Steal Auction Proceeds

CATEGORY Gaming

LOCATION <https://github.com/GuardianAudits/axis-1/blob/master/src/callbacks/liquidity/UniswapV2DTL.sol#L119>

REVIEW Main Review

DESCRIPTION

Uniswap V2 `onCreate` callback requires that the pair does not exist, but the `onSettle` callback will not revert if someone creates it before the auction settlement.

Therefore, a malicious auction seller can create the pair just before the auction ends, adds liquidity in a proportion that results in a higher pool price, settle the auction and steal most of the quote tokens from the proceeds as a refund.

RECOMMENDATION

Consider creating the UniswapV2 pair during auction settlement.

Alternatively, make sure the AuctionHouse receives both the capacity and the tokens required for liquidity up front. Be advice that some sellers will be deterred as they may need these tokens in their protocol treasury.

PROOF OF CONCEPT <https://github.com/GuardianAudits/axis-1/blob/guardian/test/guardian/tests/GuardianTestBaseline.t.sol#L1083>

RESOLUTION

Status: Resolved.

HIGH **H-06**

RESOLVED

Anchor Width Param Will DoS Auction Settlement

CATEGORY	DoS
LOCATION	https://github.com/GuardianAudits/axis-1/blob/master/src/callbacks/liquidity/BaselineV2/BaselineAxisLaunch.sol#L367
REVIEW	Main Review

DESCRIPTION

Auction seller can set some initial params for the Baseline callback, like the `anchorTickWidth`. The `onCreate` function requires this value to be between 0 and 10.

However, setting a high anchor width increases the `ANCHOR` capacity, increasing the `capacityRatio`, DoS'ing the auctions settlement.

RECOMMENDATION

Instead of having a fixed anchor width, follow the same pattern as the `BaselineInit` policy, passing the desired `floorTickL` and calculating the `anchorTickL` with:

```
int24 anchorTickL = max(activeTS - (ANCHOR_WIDTH * T_S), _floorTickL + T_S);
```

RESOLUTION

Status: Resolved.

HIGH **H-07**

RESOLVED

Lot Creation DoS Through Uniswap Pair

CATEGORY	DoS
LOCATION	UniswapV2DTL.sol: 83, UniswapV3DTL.sol: 110
REVIEW	Main Review

DESCRIPTION

Upon creating a lot which uses a DTL callback, the `onCreate` callback is invoked. For the `UniswapV2DTL` and `UniswapV3DTL` callbacks, the `__onCreate` function validates that the pair for the `baseToken` and `quoteToken` combination does not already exist.

However a malicious actor may observe the transaction to create a lot and frontrun this transaction to create a Uniswap V2 or V3 pair with the same `baseToken` and `quoteToken`. Notice that this action does not require owning either of the tokens in the pair.

As a result the `onCreate` callback will revert, causing the lot creation to revert.

RECOMMENDATION

Remove the pair existence validation in the `__onCreate` function in the both the `UniswapV2DTL` and `UniswapV3DTL` files as it is unnecessary and introduces a DoS vector.

RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/commit/d9a0fc10036b83f0aa515887795b9e6c7189f87a>

HIGH **H-08**

RESOLVED

Baseline Launches Can Not Be Done

CATEGORY DoS
LOCATION BaselineAxisLaunch.sol
REVIEW Main Review

DESCRIPTION

When the BPOOL contract is deployed, token transfers are [locked](https://github.com/0xBaseline/baseline-v2/blob/60bed78b7bee28016321ddd8c590df6c61bae6e9/src/modules/BPOOL.v1.sol#L65) and must be unlocked via policy contracts. In the underlying Baseline protocol, the `BaselineInit` policy [unlocks](https://github.com/0xBaseline/baseline-v2/blob/60bed78b7bee28016321ddd8c590df6c61bae6e9/src/policies/BaselineInit.sol#L219) transfers after launch.

The `BaselineAxisLaunch` contract serves as the `BaselineInit` policy. However, it does not have permission to call the `setTransferLock` function.

An auction with BPOOL tokens can be created since this action only requires minting and does not require a base token transfer. However, other actions such as cancelling an auction, settling an auction, or using base tokens for swaps are not possible due to transfers being locked.

RECOMMENDATION

Include the `setTransferLock.selector` in the `requestPermissions` function when configuring the policy, and unlock transfers after an auction is created.

RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/pull/11/commits/301bbb2d2fb1654c0722818abff368014510463d>

MEDIUM **M-01**

RESOLVED

discoveryTickWidth Configured Too Low

CATEGORY	Logical Error
LOCATION	BaselineAxisLaunch::L303
REVIEW	Main Review

DESCRIPTION

Baseline sets `DISCOVERY_WIDTH` to 350 in `MarketMaking.sol`, however `_onCreate()` only validates that it is set to at least one. Setting it to 350 will ensure that there is enough liquidity spread out through a range that will not cause the price of the base token to increase in an unprecedented manner. Having a larger discovery range will also mint more `bAssets`, making it harder to break through the discovery range without affecting the capacity ratio check.

The protocol allows for anchor range widths between 1 and 10, as well as discovery range widths above 0. However, in the underlying Baseline protocol, anchor and discovery range widths are fixed values of 10 and 350, respectively.

When the `sweep` or `slide` functions in the `MarketMaking` contract are called, liquidity structure will be rebalanced based on these constant values from Baseline, regardless of the initial configuration.

Additionally, reserves will be moved from the floor range to the anchor range during `sweep` in this case, which is an unwanted situation in Baseline. This happens because `sweep` function adds liquidity to anchor first, and then adds reserves to floor. However, the same amount of liquidity for a much wider anchor range will require more reserves, causing floor reserves to decrease.

Also, it is crucial for the discovery range to be wide enough and filled with BPOOL tokens in order to provide sufficient liquidity for swaps and ensure healthy price movements. A narrow discovery range could result in a single swap moving the current price tick well above the upper discovery tick, where there is minimal liquidity.

Other differences compared to Baseline are `floorReservesPercent` value being configurable (which affects [liquidity thickness](https://github.com/0xBaseline/baseline-v2/blob/60bed78b7bee28016321ddd8c590df6c61bae6e9/src/policies/MarketMaking.sol#L67C1-L69C44)), and lack of gap between floor and anchor ranges. In baseline, floor has the thickest liquidity and anchor liquidity is much lower compared to floor.

M-01 discoveryTickWidth Configured Too Low

CONTINUED

RECOMMENDATION

Set the discovery range to 350 tick spacings. Consider using the Baseline range widths instead of allowing them to be configured by the seller. Also make sure that floor range has much more reserves compared to anchor.

PROOF OF CONCEPT <https://github.com/GuardianAudits/axis-1/blob/guardian/test/guardian/tests/GuardianTestBaseline.t.sol#L1170>

RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/pull/11/commits/8e1481cb85af277616b71934377df6106b205e99>

MEDIUM **M-02**

RESOLVED

Inconsistent Prices Between Pool and Auction

CATEGORY	Validation
LOCATION	Lack Of Present Code
REVIEW	Main Review

DESCRIPTION

The seller of the `bAsset` determines the open price for the Uniswap V3 Pool when creating the `BPool`. This is done by passing `_initialActiveTick` in the constructor. Fixed-Priced Auctions are intended to sell the asset at an opening price, but there is no validation that the Auction price matches the Uniswap V3 Pool price. Since no validation occurs, a malicious seller can perform two different dishonest actions:

1. Set the price of the `bAsset` to a lower price than the Auction. This will cause any users who bought tokens in an auction to immediately incur an unrealized loss. Additionally, the seller can purchase tokens at a discount.
2. Set the price of the `bAsset` to a higher price than the Auction. In this scenario, a seller can set themselves as the curator, and sell the curator reward. This action can also push the Uniswap V3 Pool price back to the Fixed-Priced Auction price.

RECOMMENDATION

Verify that the price being initialized is the same price that is being used for Fixed-Price Auction.

RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/pull/11/commits/8b5be94680ee01a8be49058062f7ec68004d5413>

MEDIUM **M-03**

RESOLVED

Incorrect Liquidity Structure Deployed

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianAudits/axis-1/blob/master/src/callbacks/liquidity/BaselineV2/BaselineAxisLaunch.sol#L592
REVIEW	Main Review

DESCRIPTION

Baseline auction settlement will revert if the `capacityRatio` is not between 100% and 102%. This makes sure that the total capacity on the system is greater than the spot supply of `bAssets`.

However, the 102% check is specific for 1% fee tier pools. If the Baseline auction is used for a lower fee tier, the liquidity structure deployed might be invalid, but the `onSettle` callback might not revert.

This issue allows `MarketMaking.bump()` to be executed just after the liquidity is deployed, granting arbitrageurs an opportunity to extract quote tokens from the positions.

RECOMMENDATION

Consider calling `MarketMaking.bump` just after the capacity ratio check, inside a try/catch, and revert if the bump succeeds. Refer to: <https://github.com/OxBaseline/baseline-v2/blob/main/test/TestFoundation.sol#L276>

PROOF OF CONCEPT <https://github.com/GuardianAudits/axis-1/blob/guardian/test/guardian/tests/GuardianTestBaseline.t.sol#L871>

RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/commit/77eedd6578124e183c9bb95f70e6f17a32b9d813>

MEDIUM **M-04**

ACKNOWLEDGED

Auction Settlement DoS'd By Creator

CATEGORY DoS
LOCATION BaseDTL.sol: 287
REVIEW Main Review

— DESCRIPTION

In the `BaseDTL` callback contract the `onSettle` callback assumes that the auction creator has the necessary balance of base tokens and has set the correct approval to the callback. However a malicious auction creator may neglect to approve the callback contract to transfer the base tokens, therefore DoSing the settlement of the auction and preventing users from claiming their bids.

This results in user's funds being held captive during the auction period until the auction can be aborted.

— RECOMMENDATION

Consider requiring that the Axis core system or the callback contract is pre-funded with the base tokens which will be used to create the initial liquidity during settlement.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

Blacklisted Addresses Halt Auction Settlement

CATEGORY DoS
LOCATION BaseDTL.sol: 368
REVIEW Main Review

— DESCRIPTION

In the `onSettle` callback any remaining quote tokens in the contract which weren't used up due to the `proceedsUtilisationPercent` or imbalance when providing liquidity are sent back to the auction seller. However since the seller never has to directly handle the quote tokens from the auction, it is possible that the seller is blacklisted for the quote token.

Thus the `onSettle` callback will always revert when there are leftover quote tokens that would be transferred to the seller. A malicious owner of a blacklisted address could leverage this to create an auction that can never be settled using a Uniswap callback. This would lock the funds of bidders and effectively create an auction which becomes a honey pot.

— RECOMMENDATION

Consider leaving extra tokens in the callback contract and tracked in a mapping to be pulled by the seller address after settlement in a separate transaction.

Otherwise consider donating the additional quote tokens to a protocol address in the event that they cannot be transferred due to a blacklisted seller, thus allowing the `onSettle` callback to complete, similar to how GMX does this here: <https://github.com/gmx-io/gmx-synthetics/blob/1938e365dc009342aa288aa6b42fc1fd3cd9e45d/contracts/token/TokenUtils.sol#L80>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

Outdated Vesting Modules Used

CATEGORY Logical Error
LOCATION BaseDTL.sol: 351
REVIEW Main Review

DESCRIPTION

In the `_onSettle` function, if configured, a vest is created for the LP tokens created from the callback. The `linearVestingModule` is ensured to be the latest vesting module in the `_onCreate` callback when the `lotConfiguration` is set, however by the time the lot is settled the `latestVersion` or `isSunset` values are not checked.

Therefore it is possible that a lot is settled with a `linearVestingModule` that is not the latest version or is sunsetted.

In the event that a vulnerability is identified in the `linearVestingModule` the protocol will not be able to wait for the current vests to end and be settled to update the module.

RECOMMENDATION

Fetch the latest `linearVestingModule` with the `_getLatestLinearVestingModule` when settling the lot.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

console.log Present in Code

CATEGORY	Best Practices
LOCATION	BaselineAxisLaunch::L578-581 & 591
REVIEW	Main Review

— DESCRIPTION

It is inadvisable to deploy code to a live blockchain with `console.log` s present. This will waste users' gas, with no benefit added.

— RECOMMENDATION

Remove all occurrences of `console.log`.

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

RESOLVED

slide Inoperable Due To Floor Config

CATEGORY Validation
LOCATION BaselineAxisLaunch::L308
REVIEW Main Review

DESCRIPTION

When verifying the `floorReservesPercent`, the validation only checks if the value is less than or equal to 99%. This can allow a user to deploy a pool with 0% of tokens allocated to the floor. This means that even if all `bAsset` holders were to sell their tokens, the floor could never be reached. This will prevent the baseline market making feature `slide()` from being operable.

Consider the following scenario:

FLOOR 200 - 400

ANCHOR 400-600

DISCOVERY 600 - 5000

ACTIVE TICK = 500

SLIDE TICK = 500 - 200 = 300

Since 300 resides in the floor, `slide()` will not be callable.

RECOMMENDATION

Require the `floorReservesPercent` to be set to at least 50%

RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/commit/da9cf83e45f72a9999cfb308235589ee1190b3ea>

LOW **L-03**

RESOLVED

Early Unvesting Is Possible

CATEGORY	Logical Error
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Auction sellers might need to vest their LP tokens after an auction has settled. Start time and expiry time of a vesting are provided by the seller.

The protocol allows vesting start time to be before `block.timestamp` to prevent a DoS by delaying the settlements. However, this allows sellers to unvest most of their LP tokens immediately after auction.

Normal scenario (start is current timestamp):

- start: `block.timestamp`
- expiry: `block.timestamp` + 1 year
- LP amount: 100

In this 1 year vesting scenario, seller can unvest 25 tokens after 3 months.

Alternative scenario (seller sets vesting start 9 years ago):

- start: `block.timestamp` - 9 years
- expiry: `block.timestamp` + 1 year
- LP amount: 100

Here, seller can unvest 90 tokens right after settlement without waiting any time.

— RECOMMENDATION

One option might be storing the total vesting period, and updating expiry based on this period after minting derivative tokens during settlement.

Another option to consider is determining a maximum time that can be before `block.timestamp`. It would still allow malicious seller to perform this but can decrease the impact.

— RESOLUTION

Status: Resolved.

L-03 Early Unvesting Is Possible

CONTINUED

<https://github.com/Axis-Fi/axis-periphery/commit/8eeb8d6abac27db5064d4edc9327e4a073dd9165>

LOW **L-04**

RESOLVED

Unused active Flag

CATEGORY	Validation
LOCATION	BaseDTL.sol: 224
REVIEW	Main Review

— DESCRIPTION

In the `_onCancel` callback implementation the active flag is set to false for the `lotConfiguration`, however the active flag is not used to validate whether a lot is active anywhere.

— RECOMMENDATION

Although there are mechanisms in place to prevent the `onSettle` and `onCurate` functions being called for a cancelled lot, consider implementing additional safeguards on the BaseDTL callback side to prevent these callbacks from being called for cancelled lots.

— RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/commit/81e08e5d7c329d2340d4f38ab5414f9d68f1a4b6>

LOW **L-05**

ACKNOWLEDGED

Discovery Range Loose Validations

CATEGORY	Validation
LOCATION	BaselineAxisLaunch.sol: 383
REVIEW	Main Review

— DESCRIPTION

In the onCreate callback the `discoveryRangeUpper` is validated to be within the `MAX_TICK`, however if the `discoveryRangeUpper` is assigned such that it is even close to the `MAX_TICK` this can lead to a DoS of the Baseline liquidity operations down the road.

For example:

TickSpacing: 20

Discovery ticks: [MaxTick - 200, MaxTick]

A sweep cannot be triggered in this case as it would cause the discovery upper tick to exceed the max tick.

Although it is unlikely that a configuration is made where the `discoveryUpperTick` is close to the `MAX_TICK`, such a configuration should not be allowed.

— RECOMMENDATION

Consider restricting the validation on the `discoveryMaxTick` further to an expected range.

— RESOLUTION

Status: Acknowledged.

<https://github.com/Axis-Fi/axis-periphery/pull/11>

LOW **L-06**

RESOLVED

Unused Error

CATEGORY	Best Practices
LOCATION	https://github.com/GuardianAudits/axis-2/blob/85a8ec4195388df0c264e332d1d72c87a66881ea/src/callbacks/liquidity/BaselineV2/BaselineAxisLaunch.sol#L49
REVIEW	Main Review

DESCRIPTION

`Callback_Params_PoolTickMismatch` error is defined but never used.

RECOMMENDATION

Remove unused errors.

RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/pull/11/commits/b25737309223ed464688046c7a0cf7bf0555eab>

LOW **L-07**

RESOLVED

High floorReservesPercent Causes DoS

CATEGORY	Validation
LOCATION	BaselineAxisLaunch.sol: 308
REVIEW	Main Review

DESCRIPTION

In the onCreate callback the `floorReservesPercent` is validated to be no more than 99%, however this loose validation allows users to create a liquidity structure where the majority of the liquidity is allocated to the Floor position.

As a result a malicious actor is able to buy through all of the Anchor and Discovery range liquidity as it can be very thin. The malicious actor can then set an LP outside of the discovery range in order to assign the price outside of the liquidity structure.

Once the active price is allowed to exceed the Discovery range liquidity operations will be DoS'd as a sweep cannot occur.

<https://github.com/OxBaseline/baseline-v2/blob/60bed78b7bee28016321ddd8c590df6c61bae6e9/src/policies/MarketMaking.sol#L217>

RECOMMENDATION

Consider making the floorReservesPercent more strict such that it is unlikely that a liquidity structure can be deployed where a malicious actor can easily buy through all of the available liquidity to DoS the system.

PROOF OF CONCEPT <https://github.com/GuardianAudits/axis-1/pull/5/files>

RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/commit/47e9cd58071bbc4d87b470e15742a9f105b95377>

LOW **L-08**

RESOLVED

Arbitrary Byte Length Risk

CATEGORY	Warning
LOCATION	UniswapV3DTL.sol
REVIEW	Main Review

DESCRIPTION

`params.implParams` bytes are saved into storage within `_onCreate`, and operations may appear fine from the perspective of the user. During settlement callback when `mintAndDeposit` is executed, the `implParams` are loaded into memory: `(uint24 poolFee) = abi.decode(lotConfiguration[lotId].implParams, (uint24));`

The first 24 bits can be decoded properly into a `uint24`, but there is no guarantee that the bytes do not have arbitrary padding afterwards that inflate the payload. Consequently, there will be extremely large gas costs associated with the load into memory upon settlement. It's important to note that there isn't a large risk since the callbacks do not have a predetermined max gas limit, but noteworthy for future-proofing operations.

RECOMMENDATION

Consider documenting this or placing a cap on the length of the bytes.

RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/commit/04e73e93181019261cd0c8d0f425b4f526fcc860>

LOW **L-09**

ACKNOWLEDGED

Esoteric Token Pairs Are Not Supported

CATEGORY Warning
LOCATION SqrtPriceMath.sol: 34
REVIEW Main Review

DESCRIPTION

The `getSqrtPriceX96` function used for the `UniswapV3DTL` callback computes the `ratioX192` and stores it as a `uint256` before taking the square root to compute the `sqrPriceX96Temp`.

As a result with some esoteric token pairs which have a large difference in amounts, the intermediate `ratioX192` value can overflow the `uint256` size.

For example `uint160 price = SqrtPriceMath.getSqrtPriceX96(address(_baseToken), _BASELINE_QUOTE_TOKEN, 1e6, 1e26);` causes such an overflow.

Such a combination of tokens may arise when one token has low precision, such as USDC and another one has high precision such as YAMv2 or a very low price.

RECOMMENDATION

Be wary of this when supporting tokens and token prices in auctions. It is unlikely that supported token pairs would cause this issue. However it may be worth adding validations to disallow auctions which would cause this logic to revert based on token decimals and prices.

RESOLUTION

Status: Acknowledged.

LOW **L-10**

RESOLVED

Launches Restrict Liquidity Structure

CATEGORY	Validation
LOCATION	BaselineAxisLaunch.sol
REVIEW	Main Review

DESCRIPTION

In the `_onCreate` callback the validation correctly requires that the anchor range cannot exceed a width of 10 tick spacings, however the floor range upper tick is always assigned to the lower tick of the anchor range. Therefore there can be no liquidity structures with a gap in between the upper floor tick and the lower anchor tick as will commonly be the case where there is a significant premium to the baseline value.

As a result the initial liquidity structure configuration is limited in that the active price (upper tick range of the anchor position) cannot be more than 10 ticks above the floor position.

RECOMMENDATION

Consider allowing a separate configuration variable for the upper floor tick, where the liquidity structure can have a gap between the floor and the lower anchor range.

Be sure to maintain the existing Anchor width validation and add relevant validation such that the anchor cannot collide with the floor and the anchor and floor are within a reasonable distance from each other. Additionally, if the Anchor range is within 10 tick spacings of the floor range there should be no gap between the two positions.

RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/pull/11/commits/9839ce1b41dabec2d6bc90a73992fbf1234391ac>

LOW **L-11**

RESOLVED

Misleading Comment

CATEGORY Best Practices

LOCATION <https://github.com/GuardianAudits/axis-2/blob/85a8ec4195388df0c264e332d1d72c87a66881ea/src/callbacks/liquidity/BaseDTL.sol#L359>

REVIEW Main Review

DESCRIPTION

"Send the LP tokens to the seller" comment in L359 of the `BaseDTL` contract is misleading since LP tokens are transferred to the `recipient` not to the seller.

RECOMMENDATION

Consider updating the comment.

RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/commit/7521cd6863b70666428e65ec861706adf53b1571>

LOW **L-12**

RESOLVED

Seller May Receive All Proceeds

CATEGORY	Logical Error
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Both Uniswap and Baseline auction callbacks allow the owner to set a percentage of the proceeds when adding liquidity during auction settlement.

The issue relies on the validation for this percentage. Baseline auction requires this value to be within 1% and 100%, while Uniswap requires 0% to 100%.

This allows auction sellers to set a very small value, and receive most of the proceeds which were meant to be added as liquidity.

— RECOMMENDATION

Consider increasing the lower limit for the proceeds used for liquidity providing.

— RESOLUTION

Status: Resolved.

<https://github.com/Axis-Fi/axis-periphery/commit/2f1e1b81d949dcf8b80400f580a5fa8665f7f725>

HIGH **H-02**

ACKNOWLEDGED

Capacity Validation Doesn't Account For Fees

CATEGORY Logical Error
LOCATION BaselineAxisLaunch.sol
REVIEW Remediation Review

DESCRIPTION

In the onCreate callback in the `BaselineAxisLaunch` the `capacity_` value passed by the auction house is the total capacity of the auction:

<https://github.com/Axis-Fi/axis-core/blob/a9f9b26dad49eef8a3a3a92b1b99edfe37d9c64c/src/BatchAuctionHouse.sol#L95>

However in the onSettle callback the `proceeds_` value is based on the total sold less the fees taken by the protocol and referrer.

Therefore the `_onCreate` function will overestimate the amount of circulating supply which can be supported by the resulting liquidity structure upon settlement. This is because the `_onCreate` function assumes that the entire `auctionPrice * capacity_` will be the `expectedProceeds`, when in reality this amount less the fees will be split by the `poolPercent` and deployed into the liquidity structure.

RECOMMENDATION

Compute the total fees which would be taken if the entire capacity of the auction were to sell out and deduct this amount from the `expectedProceeds`. Additionally, ensure this fee amount cannot change in the time between auction creation and settlement.

RESOLUTION

Status: Acknowledged.

<https://github.com/Axis-Fi/axis-periphery/commit/dd21d0b8b8c5e1cd767f6d85f51bedd7d7b55852>

MEDIUM **M-01**

ACKNOWLEDGED

Baseline Launch Create DoS

CATEGORY	DoS
LOCATION	BaselineAxisLaunch.sol: 404, 488
REVIEW	Remediation Review

DESCRIPTION

In the onCreate callback for the `BaselineAxisLaunch` there are several checks against the active tick.

The first of which compares the result of `BP00L.getActiveTS` to the assigned Anchor upper tick on line 404.

The second compares the auctionTick to the `activeTick` to ensure that the `activeTick` is greater than or equal to the `auctionTick` so that auction bidders are not disincentivized.

Both of these validations can be manipulated to fail in the same way described in H-X from the main review. This would lead to a DoS of the creation transaction.

The risk of frontrunning to cause this DoS is limited as the Baseline launches are planned for Blast, however this issue highlights that these validations are likely not the best that could be performed in the `onCreate` function, since the actual baseline `activeTick` launch price is simply re-assigned in the settle callback based on the upper anchor tick.

RECOMMENDATION

Instead of the current validation in `onCreate`, we propose the following changes to the validations and Baseline launch tick:

- Validate that `upperAnchorTick - 1 TS <= auctionTick < upperAnchorTick` in the `onCreate` callback

This way there is no reliance on the activeTick of the pool and we know that the upperAnchorTick (which is more permanent than whatever the current active tick is at time of creation is) correctly encapsulates the auction price.

- Set the `targetSqrtPrice` to the auction price in the `settle` callback

M-01 Baseline Launch Create DoS

CONTINUED

Using the upper price of the anchor position creates a weird behavior where the result of `BP00L.getActiveTS` is no longer the `upperAnchorTick` right on launch, this is because the `getActiveTS` function rounds up to the next full tick when on an even tick spacing.

Additionally now the price can be exactly that of the auction.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Uniswap V2 Auctions Can Be DoS'd With Many Quote Tokens

CATEGORY	Warning
LOCATION	UniswapV2DTL.sol: 262
REVIEW	Remediation Review

— DESCRIPTION

In the `_mitigateDonation` function base tokens are sent to the Uniswap V2 pool and additional quote tokens are removed to assign the price as desired.

However in some rare circumstances a malicious actor may still DoS Uniswap V2 pools by donating a large amount of quote tokens to the pair.

For example:

- Consider a base token with 6 decimals and a quote token with 18 decimals
- Price is such that 1 base token == 1 quote token
- The attacker initially donates $1e24$ quote tokens to the uniswap pair
- The auction settles and the `_migrateDonation` logic sends a single wei of base tokens to the pair before syncing
- Then roughly $1e6$ base tokens are transferred in and $(1e24-1e18)$ quote tokens are requested out
- The Uniswap K value based on reserves is $1e24 * 1 = 1e24$
- The Uniswap K value after the swap at the desired reserves is $1e18 * 1e6 = 1e24$
- The swap would take 30 basis points fees and so the Uniswap K value validation would revert

This is unlikely as typically a base token within Axis will have 18 decimals and it will often be very costly to donate this amount of quote tokens.

— RECOMMENDATION

This finding serves merely as a warning to be aware of this case.

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Lacking safeTransfer Usage

CATEGORY	Best Practices
LOCATION	UniswapV2DTL.sol: 235, 252
REVIEW	Remediation Review

DESCRIPTION

When transferring the `quoteTokensToTransfer` to the Uniswap V2 pool, `transfer` is used, however the `quote` token may be a token which returns false upon transfer failure. Though it is unlikely that this transfer fails, consider using `safeTransfer` to ensure that the transfer reverts instead of silently failing.

RECOMMENDATION

`safeTransfer` should be used instead of `transfer`.

RESOLUTION

Status: Acknowledged.

<https://github.com/Axis-Fi/axis-periphery/commit/ceda80d5e3acfcc927e052143f554bbb59f16cd>

LOW **L-03**

ACKNOWLEDGED

Unnecessary totalCollatSupply

CATEGORY Optimization
LOCATION BaselineAxisLaunch.sol: 831
REVIEW Remediation Review

DESCRIPTION

The `totalCollatSupply` is deducted from the `totalSpotSupply` and then added back to the `totalSpotSupply` when computing the `capacityRatio`. The `totalSpotSupply` with the deducted `totalCollatSupply` is not used anywhere else.

RECOMMENDATION

Remove the unnecessary `totalCollatSupply` deduction and remove the subsequent addition on line 831.

RESOLUTION

Status: Acknowledged.

<https://github.com/Axis-Fi/axis-periphery/commit/dfbc90b87c4d70be2aca9ad03b988596f89e0416>

LOW **L-04**

ACKNOWLEDGED

Missing auctionComplete Check

CATEGORY Validation
LOCATION BaselineAxisLaunch.sol: 601
REVIEW Remediation Review

DESCRIPTION

In the `_onCurate` function there is no validation that the `auctionComplete` boolean is false as there is in the other callbacks.

RECOMMENDATION

Consider implementing an `auctionComplete` validation in the `_onCurate` function.

RESOLUTION

Status: Acknowledged.

<https://github.com/Axis-Fi/axis-periphery/commit/9afeb1efea927f77ed1110f0dce15451337162d8>

LOW **L-05**

ACKNOWLEDGED

Loop Vault Capacity Is Not Included

CATEGORY Warning
LOCATION BaselineAxisLaunch.sol: 818
REVIEW Remediation Review

DESCRIPTION

At the end of the `onSettle` callback validation is performed to ensure the solvency of the launched system, however this solvency check does not include the capacity from the loop vault.

This may be fine as it is unclear whether the loop vault will be a part of the Axis launches, and even if it is, would be unlikely to have any debt at the time of auction settlement.

RECOMMENDATION

Confirm whether or not the new Baseline loop vault would be included in the Axis launches, and if so, consider adding the `L00PS.totalDebt()` value when calculating the `debtCapacity`.

RESOLUTION

Status: Acknowledged.

<https://github.com/Axis-Fi/axis-periphery/commit/3e24ff7ee94b7ac7bed6482385d52d1e59c56a12>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.